

Help improve Flutter! [Take our Q2 survey](#)

[Performance](#) > Best practices



Performance best practices

How to ensure that your Flutter app is performant.

Note

To learn how to use the **Performance View** (part of Flutter DevTools) for debugging performance issues, see [Using the Performance view](#).

Generally, Flutter applications are performant by default, so you only need to avoid common pitfalls to get excellent performance. These best practice recommendations will help you write the most performant Flutter app possible.

Note

If you are writing web apps in Flutter, you might be interested in a series of articles, written by the Flutter Material team, after they modified the [Flutter Gallery](#) app to make it more performant on the web:

- [Optimizing performance in Flutter web apps with tree shaking and deferred loading](#)
- [Improving perceived performance with image placeholders, precaching, and disabled navigation transitions](#)
- [Building performant Flutter widgets](#)

How do you design a Flutter app to most efficiently render your scenes? In particular, how do you ensure that the painting code generated by the framework is as efficient as possible? Some rendering and layout operations are known to be slow, but can't always be avoided. They should be used thoughtfully, following the guidance below.

Minimize expensive operations

Some operations are more expensive than others, meaning that they consume more resources. Obviously, you want to only use these operations when necessary. How you design and implement your app's UI can have a big impact on how efficiently it runs.

Control build() cost

Here are some things to keep in mind when designing your UI:

- Avoid repetitive and costly work in `build()` methods since `build()` can be invoked frequently when ancestor widgets rebuild.
- Avoid overly large single widgets with a large `build()` function. Split them into different widgets based on encapsulation but also on how they change:
 - When `setState()` is called on a `State` object, all descendent widgets rebuild. Therefore, localize the `setState()` call to the part of the subtree whose UI actually needs to change. Avoid calling `setState()` high up in the tree if the change is contained to a small part of the tree.
 - The traversal to rebuild all descendents stops when the same instance of the child widget as the previous frame is re-encountered. This technique is heavily used inside the framework for optimizing animations where the animation doesn't affect the child subtree. See the [TransitionBuilder](#) pattern and the [source code for SlideTransition](#), which uses this principle to avoid rebuilding its descendents when animating. ("Same instance" is evaluated using `operator ==`, but see the pitfalls section at the end of this page for advice on when to avoid overriding `operator ==`.)
 - Use `const` constructors on widgets as much as possible, since they allow Flutter to short-circuit most of the rebuild work. To be automatically reminded to use `const` when possible, enable the recommended lints from the [flutter_lints](#) package. For more information, check out the [flutter_lints migration guide](#).
 - To create reusable pieces of UIs, prefer using a [StatelessWidget](#) rather than a function.

For more information, check out:

- [Performance considerations](#), part of the [StatefulWidget](#) API doc
- [Widgets vs helper methods](#), a video from the official Flutter YouTube channel that explains why widgets (especially widgets with `const` constructors) are more performant than functions.

Use StringBuffer for efficient string building

When you need to build a string from multiple parts, especially inside a loop, using the `+` operator can be inefficient because it creates a new `String` object on each concatenation. A better approach is to use `StringBuffer`, which collects all the strings and concatenates them only once, when you call `toString()`.



Use `saveLayer()` thoughtfully

Some Flutter code uses `saveLayer()`, an expensive operation, to implement various visual effects in the UI. Even if your code doesn't explicitly call `saveLayer()`, other widgets or packages that you use might call it behind the scenes. Perhaps your app is calling `saveLayer()` more than necessary; excessive calls to `saveLayer()` can cause jank.

Why is `saveLayer` expensive?

Calling `saveLayer()` allocates an offscreen buffer and drawing content into the offscreen buffer might trigger a render target switch. The GPU wants to run like a firehose, and a render target switch forces the GPU to redirect that stream temporarily and then direct it back again. On mobile GPUs this is particularly disruptive to rendering throughput.

When is `saveLayer` required?

At runtime, if you need to dynamically display various shapes coming from a server (for example), each with some transparency, that might (or might not) overlap, then you pretty much have to use `saveLayer()`.

Debugging calls to `saveLayer`

How can you tell how often your app calls `saveLayer()`, either directly or indirectly? The `saveLayer()` method triggers an event on the [DevTools timeline](#); learn when your scene uses `saveLayer` by checking the `PerformanceOverlayLayer.checkerboardOffscreenLayers` switch in the [DevTools Performance view](#).

Minimizing calls to `saveLayer`

Can you avoid calls to `saveLayer`? It might require rethinking of how you create your visual effects:

- If the calls are coming from *your* code, can you reduce or eliminate them? For example, perhaps your UI overlaps two shapes, each having non-zero transparency:
 - If they always overlap in the same amount, in the same way, with the same transparency, you can precalculate what this overlapped, semi-transparent object looks like, cache it, and use that instead of calling `saveLayer()`. This works with any static shape you can precalculate.
 - Can you refactor your painting logic to avoid overlaps altogether?
- If the calls are coming from a package that you don't own, contact the package owner and ask why these calls are necessary. Can they be reduced or eliminated? If not, you might need to find another package, or write your own.

📌 Note to package owners

As a best practice, consider providing documentation for when `saveLayer` might be necessary for your package, how it might be avoided, and when it can't be avoided.

Other widgets that might trigger `saveLayer()` and are potentially costly:

- `ShaderMask`
- `ColorFilter`
- `Chip` —might trigger a call to `saveLayer()` if `disabledColorAlpha != 0xff`
- `Text` —might trigger a call to `saveLayer()` if there's an `overflowShader`

Minimize use of opacity and clipping

Opacity is another expensive operation, as is clipping. Here are some tips you might find to be useful:

- Use the `Opacity` widget only when necessary. See the [Transparent image](#) section in the `Opacity` API page for an example of applying opacity directly to an image, which is faster than using the `Opacity` widget.
- Instead of wrapping simple shapes or text in an `Opacity` widget, it's usually faster to just draw them with a semitransparent color. (Though this only works if there are no overlapping bits in the to-be-drawn shape.)
- To implement fading in an image, consider using the `FadeInImage` widget, which applies a gradual opacity using the GPU's fragment shader. For more information, check out the [Opacity](#) docs.
- **Clipping** doesn't call `saveLayer()` (unless explicitly requested with `Clip.antiAliasWithSaveLayer`), so these operations aren't as expensive as `Opacity`, but clipping is still costly, so use with caution. By default, clipping is disabled (`Clip.none`), so you must explicitly enable it when needed.
- To create a rectangle with rounded corners, instead of applying a clipping rectangle, consider using the `borderRadius` property offered by many of the widget classes.

Implement grids and lists thoughtfully

How your grids and lists are implemented might be causing performance problems for your app. This section describes an important best practice when creating grids and lists, and how to determine whether your app uses excessive layout passes.

Be lazy!

When building a large grid or list, use the lazy builder methods, with callbacks. That ensures that only the visible portion of the screen is built at startup time.

For more information and examples, check out:

- [Working with long lists](#)
- [Creating a `ListView` that loads one page at a time](#) a community article by AbdulRahman AlHamali
- [`ListView.builder`](#) API

Avoid intrinsics

For information on how intrinsic passes might be causing problems with your grids and lists, see the next section.

Minimize layout passes caused by intrinsic operations

If you've done much Flutter programming, you are probably familiar with [how layout and constraints work](#) when creating your UI. You might even have memorized Flutter's basic layout rule: **Constraints go down. Sizes go up. Parent sets position.**

For some widgets, particularly grids and lists, the layout process can be expensive. Flutter strives to perform just one layout pass over the widgets but, sometimes, a second pass (called an *intrinsic pass*) is needed, and that can slow performance.

What is an intrinsic pass?

An intrinsic pass happens when, for example, you want all cells to have the size of the biggest or smallest cell (or some similar calculation that requires polling all cells).

For example, consider a large grid of `Card`s. A grid should have uniformly sized cells, so the layout code performs a pass, starting from the root of the grid (in the widget tree), asking **each** card in the grid (not just the visible cards) to return its *intrinsic* size—the size that the widget prefers, assuming no constraints. With this information, the framework determines a uniform cell size, and re-visits all grid cells a second time, telling each card what size to use.

Debugging intrinsic passes

To determine whether you have excessive intrinsic passes, enable the [Track layouts option](#) in DevTools (disabled by default), and look at the app's [stack trace](#) to learn how many layout passes were performed. Once you enable tracking, intrinsic timeline events are labeled as '\$runtimeType intrinsics'.

Avoiding intrinsic passes

You have a couple options for avoiding the intrinsic pass:

- Set the cells to a fixed size up front.
- Choose a particular cell to be the "anchor" cell—all cells will be sized relative to this cell. Write a custom `RenderObject` that positions the child anchor first and then lays out the other children around it.

To dive even deeper into how layout works, check out the [layout and rendering](#) section in the [Flutter architectural overview](#).

Build and display frames in 16ms

Since there are two separate threads for building and rendering, you have 16ms for building, and 16ms for rendering on a 60Hz display. If latency is a concern, build and display a frame in 16ms *or less*. Note that means built in 8ms or less, and rendered in 8ms or less, for a total of 16ms or less.

If your frames are rendering in well under 16ms total in [profile mode](#), you likely don't have to worry about performance even if some performance pitfalls apply, but you should still aim to build and render a frame as fast as possible. Why?

- Lowering the frame render time below 16ms might not make a visual difference, but it **improves battery life** and thermal issues.
- It might run fine on your device, but consider performance for the lowest device you are targeting.
- As 120fps devices become more widely available, you'll want to render frames in under 8ms (total) in order to provide the smoothest experience.

If you are wondering why 60fps leads to a smooth visual experience, check out the video [Why 60fps?](#)

Pitfalls

If you need to tune your app's performance, or perhaps the UI isn't as smooth as you expect, the [DevTools Performance view](#) can help!

Also, the Flutter plugin for your IDE might be useful. In the Flutter Performance window, enable the **Show widget rebuild information** check box. This feature helps you detect when frames are being rendered and displayed in more than 16ms. Where possible, the plugin provides a link to a relevant tip.

The following behaviors might negatively impact your app's performance.

- Avoid using the `Opacity` widget, and particularly avoid it in an animation. Use `AnimatedOpacity` or `FadeInImage` instead. For more information, check out [Performance considerations for opacity animation](#).
- When using an `AnimatedBuilder`, avoid putting a subtree in the builder function that builds widgets that don't depend on the animation. This subtree is rebuilt for every tick of the animation. Instead, build that part of the subtree once and pass it as a child to the `AnimatedBuilder`. For more information, check out [Performance optimizations](#).
- Avoid clipping in an animation. If possible, pre-clip the image before animating it.
- Avoid using constructors with a concrete `List` of children (such as `Column()` or `ListView()`) if most of the children are not visible on screen to avoid the build cost.
- Avoid overriding `operator ==` on `Widget` objects. While it might seem like it would help by avoiding unnecessary rebuilds, in practice it hurts performance because it results in $O(N^2)$ behavior. The only exception to this rule is leaf widgets (widgets with no children), in the specific case where comparing the properties of the widget is likely to be significantly more efficient than rebuilding the widget and where the widget will rarely change configuration. Even in such cases, it is generally preferable to rely on caching the widgets, because even one override of `operator ==` can result in across-the-board performance degradation as the compiler can no longer assume that the call is always static.

Resources

For more performance info, check out the following resources:

- [Performance optimizations](#) in the `AnimatedBuilder` API page
- [Performance considerations for opacity animation](#) in the `Opacity` API page
- [Child elements' lifecycle](#) and how to load them efficiently, in the `ListView` API page
- [Performance considerations](#) of a `StatefulWidget`
- [Best practices for optimizing Flutter web loading speed](#)

Was this page's content helpful?





Except as otherwise noted, this site is licensed under a [Creative Commons Attribution 4.0 International License](#), and code samples are licensed under the [3-Clause BSD License](#).

[Terms](#)[Brand](#)[Privacy](#)[Security](#)

Built with
Jaspr